Ex 3: Generative Adversarial Networks (GANs) for Synthetic Image Generation

Learning Objective:

Implementation of Generative Adversarial Networks (GANs) for Synthetic Image Generation using the MNIST Dataset.

Steps:

1. Data Preparation

- **Preprocessing:** Load MNIST, scale pixels to $[-1, 1]$ for tanh compatibility, shuffle, and batch the data.

2. The Generator (The Forger)

- **Architecture:** Projects a 100D noise vector into a $7 \times 7 \times 256$ tensor, then upsamples via Conv2DTranspose to reach the final $28 \times 28 \times 1$ image size using tanh.

3. The Discriminator (The Critic)

- **Architecture:** Uses convolutional layers with LeakyReLU and Dropout to downsample the image into a single probability score via a Sigmoid activation.

4. Loss Functions

- **Objective:** Uses Binary Cross-Entropy to penalize the Discriminator for misclassifying images and the Generator for failing to "fool" the critic.

5. Optimizers

- **Setup:** Two separate Adam optimizers are used to independently update the weights of the Generator and Discriminator.

6. Training Step

- **Execution:** Noise is fed to the Generator to create fakes; the Discriminator evaluates both fakes and reals, and gradients are applied to improve both simultaneously.

7. Training Loop

- **Iteration:** The process repeats across epochs, with periodic image generation to monitor the "learning" progress of the Generator.

8. Visualization

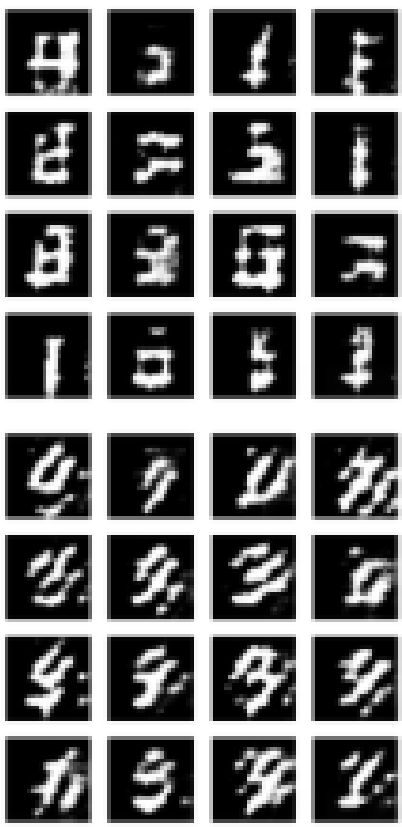
- **Comparison:** Final generated samples are plotted alongside original MNIST digits to evaluate how realistically the model has learned the data distribution.

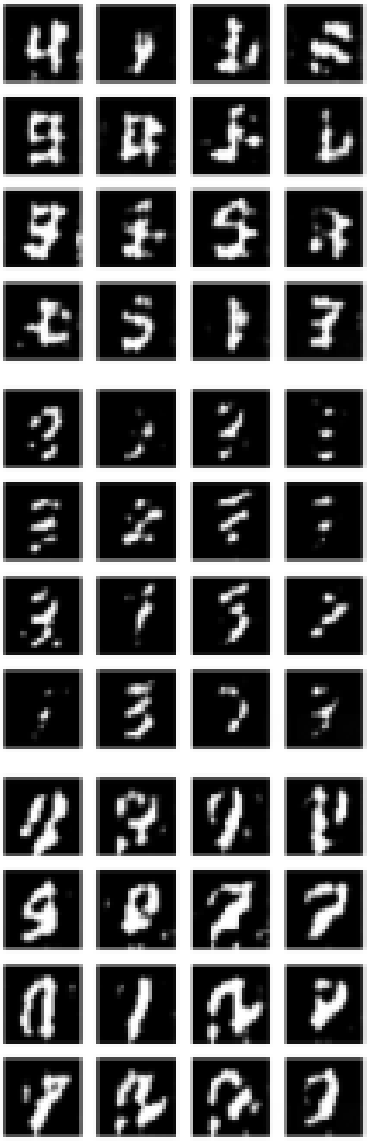
CODE:

```
import tensorflow as tf
from tensorflow.keras import layers
import matplotlib.pyplot as plt
import numpy as np

(x_train, _), (_, _) = tf.keras.datasets.mnist.load_data()
x_train = (x_train - 127.5) / 127.5
x_train = x_train.reshape(-1,28,28,1)

BUFFER_SIZE = 60000
BATCH_SIZE = 256
dataset =
tf.data.Dataset.from_tensor_slices(x_train).shuffle(BUFFER_SIZE).batch(BATCH_SIZE)

def build_generator():
    model = tf.keras.Sequential()
    model.add(layers.Dense(7*7*256, input_shape=(100,)))
    model.add(layers.Reshape((7,7,256)))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())

    model.add(layers.Conv2DTranspose(128,5,strides=1,padding='same'))
    model.add(layers.LeakyReLU())

    model.add(layers.Conv2DTranspose(64,5,strides=2,padding='same'))
    model.add(layers.LeakyReLU())

    model.add(layers.Conv2DTranspose(1,5,strides=2,padding='same',activation='tanh'))
    return model

def build_discriminator():
    model = tf.keras.Sequential()
    model.add(layers.Conv2D(64,5,strides=2,padding='same',input_shape=[28,28,1]))
    model.add(layers.LeakyReLU())
    model.add(layers.Dropout(0.3))

    model.add(layers.Conv2D(128,5,strides=2,padding='same'))
    model.add(layers.LeakyReLU())
    model.add(layers.Dropout(0.3))

    model.add(layers.Flatten())
    model.add(layers.Dense(1,activation='sigmoid'))
    return model

generator = build_generator()
```

```
discriminator = build_discriminator()

cross_entropy = tf.keras.losses.BinaryCrossentropy()

g_optimizer = tf.keras.optimizers.Adam(0.0002)
d_optimizer = tf.keras.optimizers.Adam(0.0002)

@tf.function
def train_step(images):
    noise = tf.random.normal([BATCH_SIZE,100])

    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:

        generated_images = generator(noise)

        real_output = discriminator(images)
        fake_output = discriminator(generated_images)

        gen_loss = cross_entropy(tf.ones_like(fake_output), fake_output)

        real_loss = cross_entropy(tf.ones_like(real_output), real_output)
        fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_output)
        disc_loss = real_loss + fake_loss

    gradients_gen = gen_tape.gradient(gen_loss, generator.trainable_variables)
    gradients_disc = disc_tape.gradient(disc_loss, discriminator.trainable_variables)

    g_optimizer.apply_gradients(zip(gradients_gen, generator.trainable_variables))
    d_optimizer.apply_gradients(zip(gradients_disc, discriminator.trainable_variables))

EPOCHS = 5

for epoch in range(EPOCHS):
    for image_batch in dataset:
        train_step(image_batch)

    noise = tf.random.normal([1,100])
    generated_image = generator(noise, training=False)

    plt.imshow(generated_image[0,:,:,0], cmap='gray')
    plt.title(f"Epoch {epoch+1}")
    plt.show()
```

OUTPUT:





Learning Outcome:

Upon completion of this experiment, the bigram and trigram probability distributions for next-word prediction was implemented and evaluated using the Brown Corpus.